

Aprendizaje Supervisado

Clase 23: Redes Neuronales, Evaluación y Pipelines

Dr. José Bavio

Departamento de Matemática
Universidad Nacional del Sur

2026

Contenido de la clase

- 1 Redes Neuronales Artificiales
- 2 Métricas de Evaluación
- 3 Clases Desbalanceadas y SMOTE
- 4 Pipelines y Grid Search

La neurona artificial

Idea

Una neurona recibe p entradas, las combina linealmente y aplica una función no lineal:

$$a = \sigma \left(\sum_{j=1}^p w_j x_j + b \right) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

Analogía biológica (solo para intuición)

En el cerebro, una neurona “se activa” si la suma de señales que recibe supera un umbral. La función de activación σ cumple el mismo rol: decide si la neurona “se prende” o no.

La analogía no es perfecta, pero ayuda a recordar la estructura.

Perceptrón Multicapa (MLP)

Definición

Un **MLP** (*Multilayer Perceptron*) apila capas de neuronas:

- **Capa de entrada:** dimensión p (una neurona por feature).
- **Capas ocultas:** transformaciones no lineales sucesivas.
- **Capa de salida:** produce la predicción final.

Salida de la capa l :

$$h^{(l)} = \sigma^{(l)} \left(W^{(l)} h^{(l-1)} + b^{(l)} \right)$$

¿Por qué capas ocultas?

Cada capa aprende representaciones más abstractas de los datos. La primera capa puede detectar bordes; la segunda, formas; la tercera, objetos. Con suficientes capas y neuronas, un MLP puede aproximar cualquier función continua (*teorema de aproximación universal*).

Nombre	Fórmula	Rango	Uso
Sigmoide	$1/(1 + e^{-z})$	$(0, 1)$	Salida binaria
Tanh	$\tanh(z)$	$(-1, 1)$	Capas ocultas
ReLU	$\max(0, z)$	$[0, \infty)$	Estándar hoy
Leaky ReLU	$\max(\alpha z, z)$	$\mathbb{R}$	Evita neuronas muertas
Softmax	$e^{z_k} / \sum_j e^{z_j}$	$(0, 1)^K$	Salida multiclase

¿Por qué ReLU domina hoy?

La sigmoide y la tanh “aplastan” los gradientes en sus extremos (gradiente ≈ 0): el entrenamiento se detiene. ReLU no tiene ese problema. Además es muy simple de calcular: $\max(0, z)$.

Entrenamiento: descenso por gradiente

Los parámetros $\theta = \{W^{(l)}, b^{(l)}\}$ se ajustan minimizando la función de pérdida.

Descenso por gradiente estocástico (SGD):

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

donde η es la **tasa de aprendizaje** (*learning rate*).

Intuición

Imaginar que estás en una montaña con niebla y querés bajar al valle. El gradiente te dice la dirección de máxima subida; ir en la dirección opuesta (descenso por gradiente) te lleva hacia abajo. El learning rate es cuán grande es cada paso que das.

Cuidado con η

η muy grande: el modelo “salta” y no converge.

η muy pequeño: convergencia lentísima.

Backpropagation

¿Cómo calcular $\nabla_{\theta} \mathcal{L}$?

Backpropagation aplica la regla de la cadena capa por capa, de la salida hacia la entrada, calculando eficientemente el gradiente respecto a cada parámetro de la red.

Aclaración importante

Backprop no es el algoritmo de optimización: es el método para calcular el gradiente. El algoritmo de optimización (SGD, Adam, etc.) usa ese gradiente.

Es como la diferencia entre calcular la pendiente de una montaña (backprop) y la estrategia de descenso que usás (SGD, Adam).

Adam (Kingma & Ba, 2015) es el optimizador más usado hoy: combina momentum con adaptación individual de la tasa por parámetro.

Regularización en redes neuronales

Dropout

Durante el entrenamiento, cada neurona se desactiva con probabilidad p_{drop} . Fuerza redundancia: la red no puede depender de ninguna neurona individual.

Como estudiar para el examen sin saber de antemano qué páginas del libro van a estar disponibles.

Early stopping

Entrenar hasta que el error de validación deja de mejorar. Guardar los pesos del mejor epoch.

Parar de estudiar cuando ya no aprendés más, antes de que empieces a “aprender” los ruidos.

Otras técnicas

- Regularización L_2 (*weight decay*): penaliza pesos grandes.
- Batch normalization: normaliza activaciones, acelera el entrenamiento.

Matriz de confusión

Para clasificación binaria (positivo / negativo):

		Predicho	
		Positivo	Negativo
Real	Positivo	VP	FN
	Negativo	FP	VN

- **VP** (verdadero positivo): predijo positivo y era positivo. Bien.
- **VN** (verdadero negativo): predijo negativo y era negativo. Bien.
- **FP** (falso positivo): predijo positivo pero era negativo. *Error tipo I.*
- **FN** (falso negativo): predijo negativo pero era positivo. *Error tipo II.*

$$\text{Accuracy} = \frac{VP + VN}{n}$$

$$\text{Precisión} = \frac{VP}{VP + FP}$$

$$\text{Recall} = \frac{VP}{VP + FN}$$

$$F_1 = 2 \cdot \frac{\text{Precisión} \times \text{Recall}}{\text{Precisión} + \text{Recall}}$$

Intuición de Precisión y Recall

Precisión: “De todos los que el modelo acusó de positivos, ¿cuántos realmente lo eran?”

Recall: “De todos los positivos reales, ¿cuántos detectó el modelo?”

En diagnóstico médico: recall alto es crítico (no quiero perder enfermos).

En correo spam: precisión alta importa más (no quiero tirar mail legítimo).

El problema de la accuracy con clases desbalanceadas

Ejemplo: detección de fraude

99% de las transacciones son legítimas, 1% son fraude.

Un clasificador que predice SIEMPRE “no fraude” tiene $\text{accuracy} = 99\%$.

Sin embargo, **no detecta ningún fraude**.

Soluciones

Con clases desbalanceadas, preferir:

- F_1 o F_β (con $\beta > 1$ si recall importa más)
- AUC-ROC (Area Under the ROC Curve)
- AUC-PR (Area Under the Precision-Recall Curve)

Curva ROC y AUC

La **curva ROC** grafica:

- Eje y: TPR (Recall, Sensibilidad) = $VP/(VP + FN)$
- Eje x: FPR = $FP/(FP + VN)$

para todos los posibles umbrales $\tau \in [0, 1]$.

AUC-ROC

- $AUC = 1$: clasificador perfecto.
- $AUC = 0.5$: clasificador aleatorio (diagonal).
- $AUC < 0.5$: peor que aleatorio.

Interpretación probabilística

AUC = probabilidad de que el modelo asigne un score más alto a un positivo aleatorio que a un negativo aleatorio.

No depende del umbral de decisión: mide la capacidad discriminante global.

¿Por qué el desbalance es un problema?

Efecto sobre el entrenamiento

Si el 95% de los datos son clase mayoritaria, el modelo aprende rápido a ignorar la clase minoritaria: el error en la mayoría domina la función de pérdida. El modelo “toma el atajo” de predecir siempre la clase mayoritaria.

Estrategias principales:

- 1 Resampling (oversampling / undersampling)
- 2 Ajuste de pesos de clase (`class_weight`)

SMOTE (*Synthetic Minority Oversampling Technique*, Chawla et al., 2002) genera observaciones **sintéticas** de la clase minoritaria:

- 1 Para cada punto x_i de la clase minoritaria, encontrar sus k vecinos más cercanos.
- 2 Generar un punto sintético interpolando:

$$\tilde{x} = x_i + \lambda(x_{nn} - x_i), \quad \lambda \sim \mathcal{U}(0, 1)$$

Coloquialmente

En lugar de copiar y pegar los mismos ejemplos minoritarios (oversampling simple), SMOTE “inventa” ejemplos nuevos que están entre dos ejemplos reales de la clase. Añade variedad sin salir de la región donde ya existen ejemplos.

Regla fundamental

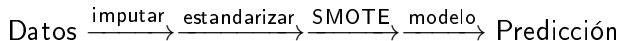
SMOTE se aplica **solo sobre el conjunto de entrenamiento**. Nunca antes de dividir los datos: eso produce data leakage.

¿Qué es un pipeline?

Definición

Un **pipeline** encadena pasos de preprocesamiento y modelado en un único objeto. Cada paso transforma los datos y los pasa al siguiente.

Ejemplo típico:



¿Por qué es importante?

Sin pipeline, es fácil cometer el error de:

- 1 Estandarizar todos los datos.
- 2 *Luego* dividir en train/test.

Resultado: el test set “contaminó” los parámetros de la estandarización. Eso es **data leakage**: el modelo vio información del test durante el entrenamiento.

Data Leakage

El error más silencioso del ML

El **data leakage** ocurre cuando información del futuro o del test set contamina el entrenamiento del modelo.

El modelo parece funcionar muy bien en evaluación, pero falla en producción.

Formas comunes de leakage

- Normalizar/estandarizar con estadísticos calculados sobre todo el dataset.
- Aplicar SMOTE antes de dividir train/test.
- Seleccionar features con toda la información antes de dividir.
- Usar features que “vienen del futuro” (ej.: resultado de un evento para predecirlo).

Solución

Usar pipelines con validación cruzada: cada transformación se ajusta solo con los datos de entrenamiento de cada fold.

Grid Search y búsqueda de hiperparámetros

Grid Search

Define una grilla de valores para cada hiperparámetro y evalúa todas las combinaciones usando validación cruzada sobre el conjunto de entrenamiento.

Método	Descripción
Grid Search	Todas las combinaciones (exhaustivo, costoso)
Random Search	Muestrea combinaciones al azar (más eficiente)
Bayesiano	Guía la búsqueda con un modelo probabilístico

Regla de oro

La evaluación con grid search usa el **conjunto de entrenamiento** (con CV interna). El conjunto de prueba queda reservado para la evaluación final. Mirar el test set durante la búsqueda de hiperparámetros invalida su uso como estimador imparcial.

Pipeline + Grid Search: la combinación correcta

Con GridSearchCV en scikit-learn sobre un pipeline:

- 1 En cada fold de CV:
 - Los pasos del pipeline se **ajustan** sobre el fold de entrenamiento.
 - Se **aplican** sobre el fold de validación.
- 2 Se elige el conjunto de hiperparámetros con mejor CV.
- 3 Se re-entrena el pipeline completo con todos los datos de entrenamiento.
- 4 Se evalúa **una sola vez** sobre el test set.

Coloquialmente

Es el equivalente a preparar el examen con ejercicios de práctica sin mirar las respuestas del examen real hasta el día del examen. Una sola vez. Sin volver a estudiar después de ver el resultado.

Modelo	Fortaleza	Limitación clave
Árbol de decisión	Interpretable	Alta varianza
Random Forest	Robusto, OOB	Caja negra
SVM	Alta dimensión	Costoso en n grande
KNN	Sin entrenamiento	Maldición dimensionalidad
MLP	Muy flexible	Requiere muchos datos

Temas transversales:

- Bias-variance tradeoff y validación cruzada (siempre).
- Métricas apropiadas al problema (accuracy no siempre sirve).
- Pipelines para evitar data leakage.
- SMOTE para clases desbalanceadas.